Clustered Smart Home Alarm System - Report

Martin Tomassi

1 Problem analysis

In the previous assignment, the whole actor tree lived inside a single `ActorSystem` and the control unit's reference was wired once, at startup, directly into every peripheral. Under that assumption, a reference to the control unit was always valid, always reachable and never changed for the entire lifetime of the process, so the finite state machine (FSM) logic could be designed without ever questioning whether the recipient actor was actually active. Running the same system in a distributed way removes this guarantee and introduces a number of problems that did not exist before.

Since the peripherals and the control unit can now start up on different nodes of the cluster, it is no longer possible to assign a reference at the time of actor creation: the system must be able to dynamically identify the control unit as soon as it becomes available anywhere in the cluster. Even after being discovered, a reference to the control unit is not permanently valid: if the control unit fails, its old reference becomes unusable, and any peripheral that depended on it must detect this and re-discover the new reference. Interactions that previously relied on local in-memory message passing, now take place over the network between separate JVMs, introducing message serialization and network latency. Also, a restarted control unit cannot assume that its state has survived. Therefore, it can neither resume operation of the state machine from where it left off, nor safely set the default state to Armed or Disarmed. Since the sensors and the keypad continue to operate regardless of the control unit's lifecycle, events generated while the control unit is unreachable or recovering must be assigned a well-defined outcome.

2 General strategy

The FSM logic, from the previous assignment, is preserved unchanged: the distributed version does not redesign the alarm system logic, it adapts the architecture to the problems identified above while adding one new state dedicated to failure recovery.

First, the system is split into independent, role-specific nodes (control unit, keypad, sensors) that join the same cluster, so no single component needs to know the physical location of the others in advance. Instead of being wired at creation time, peripherals locate the control unit through Pekko's `Receptionist` (actor discovery mechanism) and keep listening for updates, which lets them automatically re-bind to a new control unit whenever the previously known one fails. Furthermore, the control unit is supervised by its guardian, which detects its termination and restarts it into a dedicated recovery state rather than resuming normal operation. In this state, the system treats itself as neither armed nor disarmed, ignores arming requests and sensor activity, and only returns to normal operation once a valid PIN is entered. Finally, all messages exchanged between peripherals and the control unit are made serializable, so that the same asynchronous message-passing style used locally in the previous assignment continues to work transparently once actors run on different nodes.